Simple Content Provider

Added in version	1.0.3
Status	Up-to date

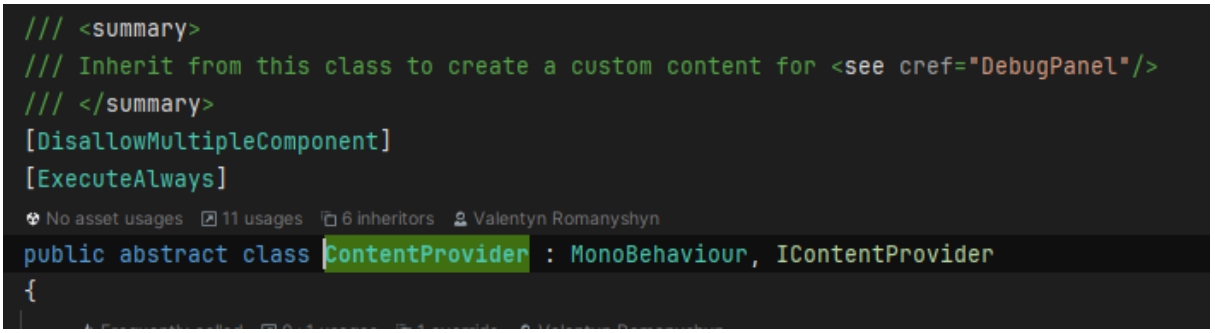
What is content provider
- In editor

Sample overview

All controls
- Groups
- Tabs
- Button
- RepeatButton
- Toggle
- Dropdown
- Label

What is content provider

`ContentProvider` is a base class for a content of a DebugPanel. It populates content for panel and links it with your games systems.



Its main method is `PopulateContent(DebugPanel panel)`

```
/// <summary>
/// <see cref="DebugPanel"/> calls it re-build UI, put all your custom code for UI here.
/// Use <see cref="DebugPanel.BeginGroup(GroupOptions, bool)"/>,
/// <see cref="DebugPanel.EndGroup"/>, <see cref="DebugPanel.AddButton"/>, and others to populate content.
/// Or helper scopes like <see cref="TabGroup"/> and <see cref="Group"/>
/// </summary>
Frequently called  3 usages  7 implementations  Valentyn Romanyshyn
void PopulateContent(DebugPanel panel);

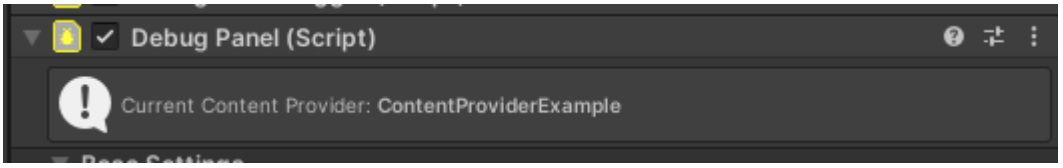
/// <inheritdoc cref="PopulateContent"/>
/// <remarks>
/// Called whenever a screen aspect ratio is more horizontal (landscape) than vertical
/// </remarks>
Frequently called  2 usages  4 implementations  Valentyn Romanyshyn
void PopulateHorizontalContent(DebugPanel panel);

/// <inheritdoc cref="PopulateContent"/>
/// <remarks>
/// Called whenever a screen aspect ratio is more vertical (portrait) than horizontal
/// </remarks>
Frequently called  2 usages  4 implementations  Valentyn Romanyshyn
void PopulateVerticalContent(DebugPanel panel);
```

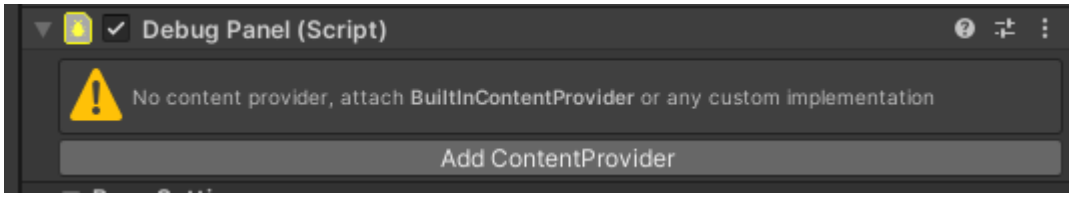
> There is also horizontal and vertical populate methods for different layouts depending on aspect ratio, see more in Horizontal And Vertical Layout

In editor

Debug panel should have a content provider, it will show the current provider in its editor



And if no content provider found will suggest to attach one



Sample overview

Sample provider script `ContentProviderExample.cs`

Here is all of the code for content creation is defined.

As you can see from first lines there is few concepts, the groups and the buttons presented. Groups is similar to IMGUI in editor, where you need to being/end group calls, and its content in between.

Using `panel.BeginGroup()` / `panel.EndGroup()` is one way to define a group.

Use `panel.AddButton()` to add simples panel ui element - button, with name and click callback.

```
/// <summary>
/// Two options for creating presented here.
/// </summary>
Frequently called  0+7 usages
public override void PopulateContent(DebugPanel.Runtime.DebugPanel panel)
{
    // Regular methods
    // BeginGroup need to be called manually at the start and EndGroup at the end
    panel.BeginGroup(GroupOptions.Vertical, groupName: "Resources");
    {
        // Regular horizontal group
        panel.BeginGroup(GroupOptions.Horizontal, groupName: "Coal");
        {
            panel.AddButton(text: "+1", () => Debug.Log(message: 1));
            panel.AddButton(text: "+5", () => Debug.Log(message: 5));
            panel.AddButton(text: "+10", () => Debug.Log(message: 10));
            panel.AddButton(text: "+20", () => Debug.Log(message: 20));
            panel.AddButton(text: "+50", () => Debug.Log(message: 50));
        }
        panel.EndGroup();
```

Below there is a way to define group with scope `struct Group : IDisposable` utilizing using statement. This makes sure you don’t forget to end the group.

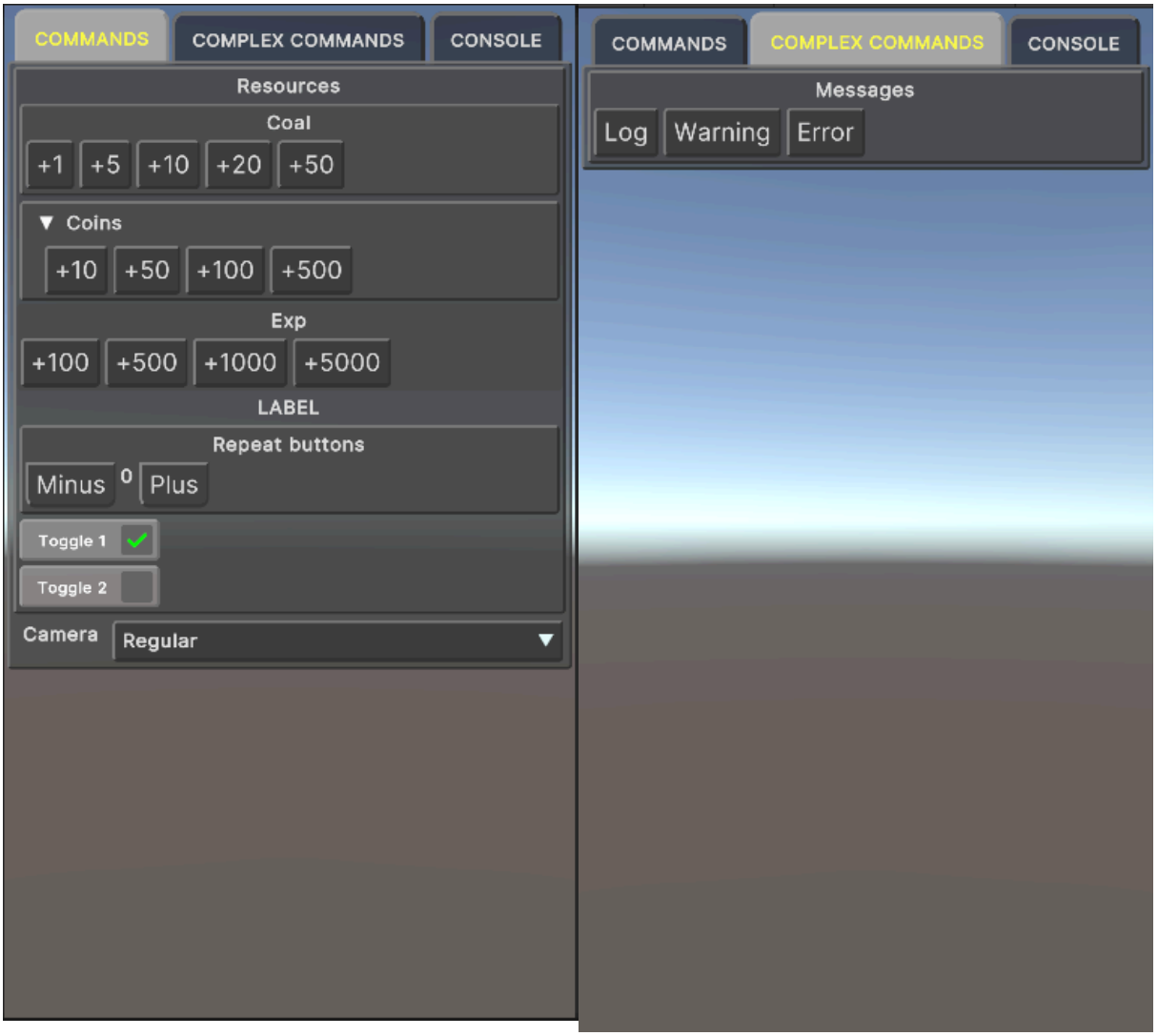
```
// Repeat buttons
using (new Group(panel, GroupOptions.Horizontal, groupName: "Repeat buttons"))
{
    panel.AddRepeatButton(text: "Minus", () =>
    {
        _repeatButtonNumber--;
        _repeatButtonLabel.text = _repeatButtonNumber.ToString(CultureInfo.InvariantCulture);
    });

    _repeatButtonLabel = panel.AddLabel(text: "0");

    panel.AddRepeatButton(text: "Plus", () =>
    {
        _repeatButtonNumber++;
        _repeatButtonLabel.text = _repeatButtonNumber.ToString(CultureInfo.InvariantCulture);
    });
}
```

All the sample code results into

Commands tab **And Complex commands tab**



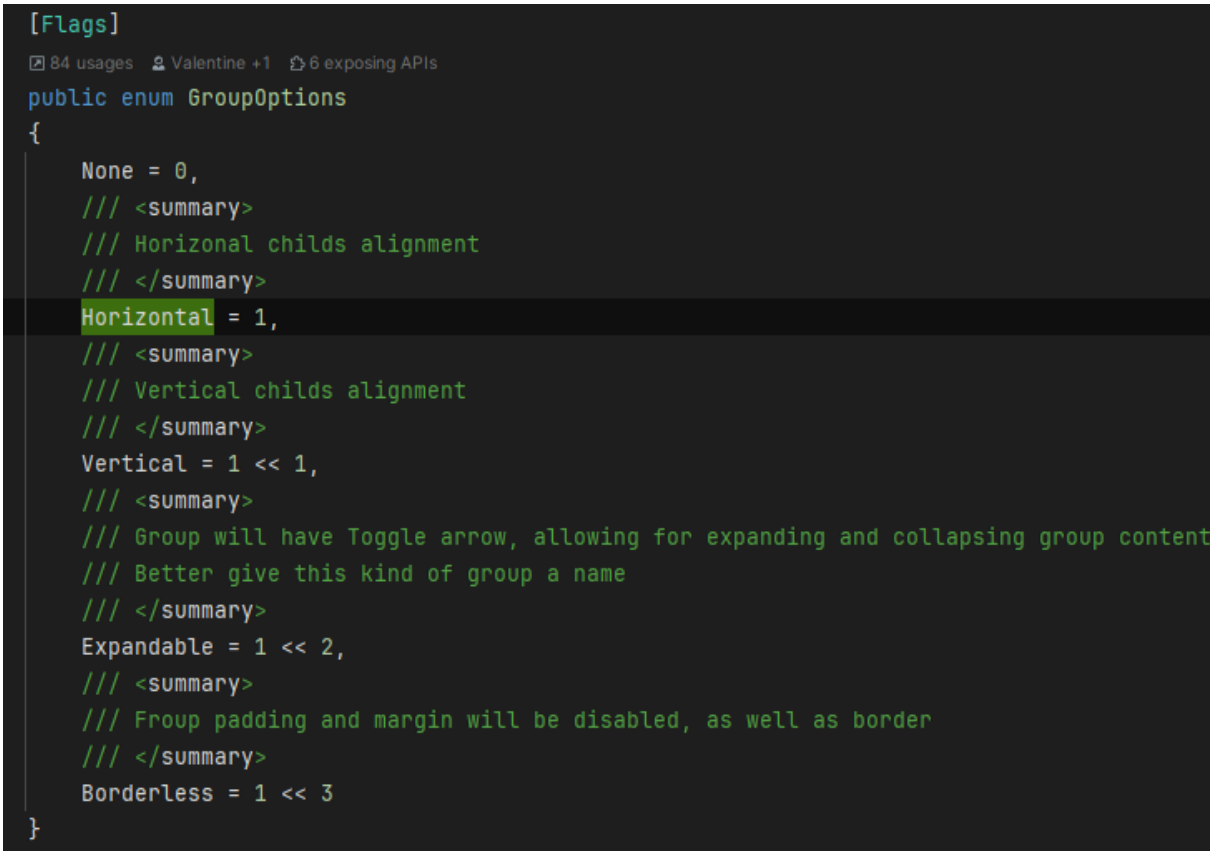
All controls

Debug panel provides a few ui elements, lets go over them.

Groups

We already mentioned groups, group is a container for other content, this content can be other groups.

Groups has a few flags that changes its behaviour/appearance



- Horizontal - aligns childs horizontally
- Vertical - aligns childs vertically
- Expandable - adds toggle to panel name, allowing to collapse it
- Borderless - removes borders and margins, for cases when panel needs to only be used for layout purposes

Group can be created with `panel.BeginGroup()` / `panel.EndGroup()` or `struct Group : IDisposable`. Group takes `GroupOptions` and `Name` for label on its top, label will only be displayed for non Borderless group.

Tabs

New tabs can be defined similar to Groups, but this needs to happen in root scope, when all of your groups is closed.

Can be done with similar `struct TabGroup : IDisposable` or corresponding `panel.BeginTabGroup()` / `panel.EndTabGroup()`.

```
// Helper structs and using block
// in this case .EndGroup/.EndTabGroup will be called at the end of using block
using (new TabGroup(panel, tabName: "COMPLEX COMMANDS"))
{
    using (new Group(panel, GroupOptions.Horizontal, groupName: "Messages"))
    {
        panel.AddButton( text: "Log", () =>
            Debug.Log("lollllllllllllllllllllllasdasdasdasddddddddddddddddddddddddd
        panel.AddButton( text: "Warning", () =>
            Debug.LogWarning("Kekasddddddddddddddddddddddddddddasdasasasasasasasasasas
        panel.AddButton( text: "Error", () =>
            Debug.LogError("Cheburekdasdasdadasddddddddddddddddddddddddddddddasddddddd
    }
```

Tabs will be added at the top of panel, there always will be at least one tab in panel - COMMANDS (main tab), optional CONSOLE tab, and between them any custom defined tabs.



Groups creates base structure for end node controls, lets go over the latter.

Button

`panel.AddButton()` - will add Button with label and click callback.

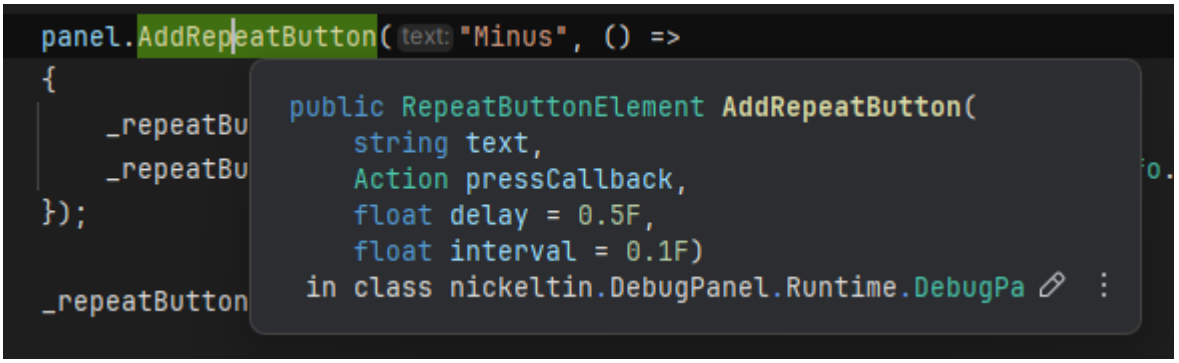
All create methods will return produced VisualElement, that can be saved to variable and accessed/modified later.

```
panel.AddButton( text: "+1", () => Debug.Log( message: 1));
panel.AddBu
panel.AddBu
panel.AddBu
```

```
public Button AddButton(string text, Action pressCallback)
 in class nickeltin.DebugPanel.Runtime.DebugPanel
```

RepeatButton

`panel.AddRepeatButton()` - similar to button, but its continuous press will trigger click callback after first click each 0.1 seconds, 10 times a second (by default).

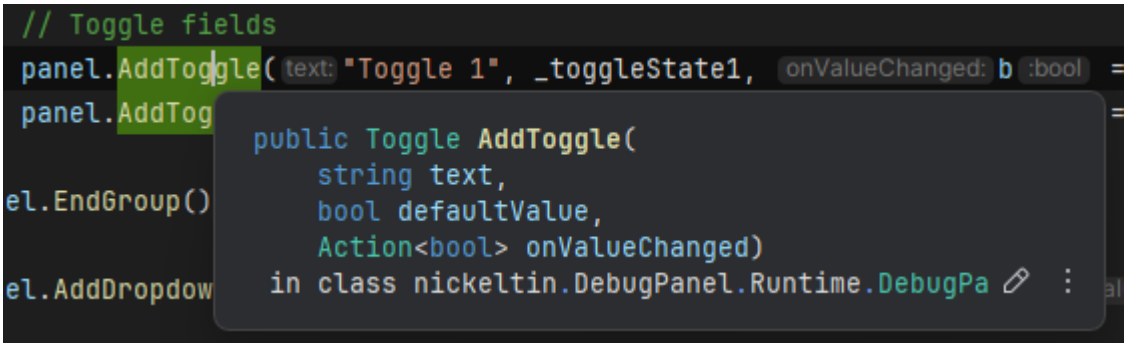


Toggle

`panel.AddToggle()` - toggle, checkbox, bool field.

Takes label text, true/false value at creation and value changed callback.

It's your responsibility to change toggle value if it's binded to your game variable, as well as update variable value from `onValueChanged` callback.

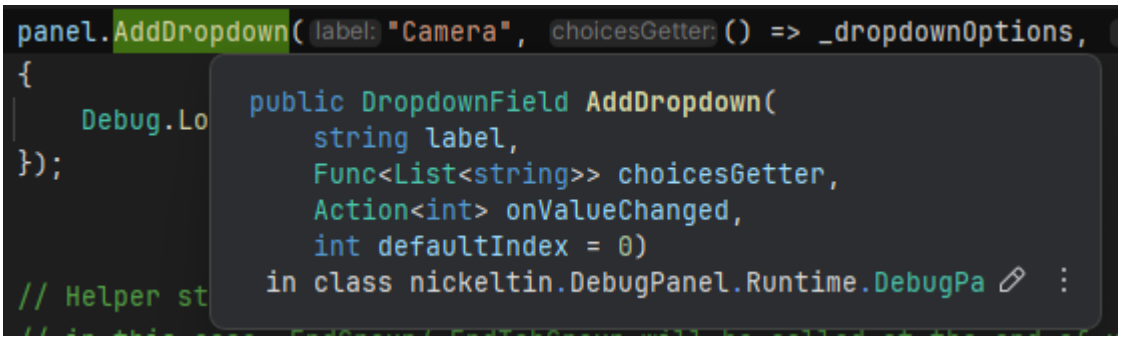


Dropdown

`panel.AddDropdown()` - dropdown with label text and provided options.

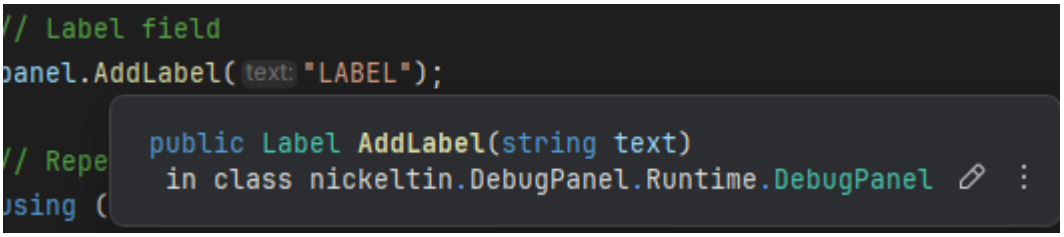
Supports dynamic options, therefore takes Func for options creation, pass here the method that provides your options.

Update binding in game variable in `onValueChanged` callback, responsibility for correct values update is also on you.



Label

`panel.AddLabel()` - simplest control, label is just piece of text. Mostly usable when you want to create dynamic label and change its value according to game state, like in this sample `_repeatButtonLabel`. For extended dynamic game state tracking in debug panel see 🏷️ Tracking Game State With Labels



Except dynamic labels there is not much use for them, since group names gives more than enough info about structure.

> 💡 Other controls will be added in next updates. Controls in plans:
>
> - Modal windows
> - Sliders
> - Lists